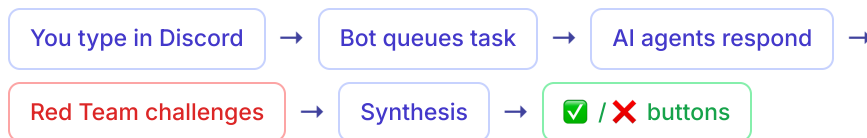


What is this? Agent Factory is a Discord bot that routes your questions to multiple AI models — Claude, Gemini, local models, and more. Ask anything in Discord, watch the agents debate, and approve or reject the outcome with a button click. No per-query API costs. Uses subscription CLIs and local models.

0 What you'll end up with



Every debate has a built-in **Red Team** agent that plays devil's advocate — attacks the dominant view to prevent echo chambers. The synthesizer tells you which objections were valid.

1 Prerequisites checklist

- ☐ **Node.js 18+** — `node --version` · Install at nodejs.org if needed
- ☐ **Discord account** — and a server where you're admin (or create one)
- ☐ **Database** — Docker Desktop (easiest) *or* a free Supabase account (supabase.com)
- ☐ **At least one AI agent** — see section 2 below
- ☐ **Claude Code installed** — for the interactive setup skill (optional but recommended)

2 Choose your agents (pick at least one)

Goal: no per-query API costs. Use subscription CLIs and local models.

 **Claude** SUBSCRIPTION
CLI
Requires **Claude Max** subscription (claude.ai). After

 **Gemini CLI** FREE
Google account auth — no API key. Install: `npm i -g @google/gemini-cli` then `gemini auth` (one-time browser login).

installing Claude Code, the `claude` binary is already there — zero extra setup.

Ollama (local) LOCAL

Runs on your machine. Install at ollama.ai, then `ollama pull llama3.2`. Needs 8GB+ RAM. Free, private, slower.

Codex CLI SUBSCRIPTION

Requires OpenAI subscription + Codex desktop app. Mac: binary at [/Applications/Codex.app/Contents/Resources/codex](https://Applications/Codex.app/Contents/Resources/codex)

Note: Configure only agents you have. Unconfigured agents are silently skipped. The bot works with just one.

3 **Discord bot setup (~5 min)**

Create the bot

- 1 **Go to discord.com/developers/applications**
Click **New Application** → give it a name (e.g. "MyAgentBot") → Create
- 2 **Add a bot**
Left sidebar → **Bot** → click **Add Bot** → confirm
- 3 **Enable Message Content Intent**
Still in Bot settings → scroll to **Privileged Gateway Intents** → toggle **Message Content Intent** ON → Save Changes.

Don't skip this. Without it the bot sees reactions but not message text.

- 4 **Copy the bot token**
Token section → **Reset Token** → copy it. Save it — you'll add it to `.env` as `DISCORD_BOT_TOKEN`.

Invite bot to your server

- 5 **Generate invite URL**
Left sidebar → **OAuth2** → **URL Generator** → Scopes: check `bot` → Permissions: `Send Messages`, `Read Messages`, `Add Reactions`, `Embed Links`, `Use Slash Commands` → copy URL → open in browser → select server → Authorize
- 6 **Get channel ID + webhook**
Enable Developer Mode (User Settings → Advanced). Right-click your channel → **Copy Channel ID** → save as `DISCORD_TASK_CHANNEL_ID`.
Then: Edit Channel → Integrations → Webhooks → New Webhook → Copy URL → save as `DISCORD_WEBHOOK_URL`.

4 Database (pick one)

Option A — Docker (local)

Requires Docker Desktop running.

```
cd ai-skills/agent-factory
docker compose up -d
```

DATABASE_URL will be:

```
postgresql://agent:agentfactory@localhost:5432/agent_factory
```

Option B — Supabase (cloud)

Free tier at supabase.com. No Docker needed.

1. New Project → set password
2. Settings → Database → Connection String (URI)
3. Copy the

```
postgresql:// ...
```

 string
4. Run:

```
psql "$DB_URL" -f schema.sql
```

5 Install & configure

```
# Clone the repo
git clone https://github.com/LNS-Research/ai-skills.git
cd ai-skills/agent-factory
npm install
```

```
# Copy the env template and fill it in
cp .env.example .env
nano .env # or open in any editor
```

Minimum `.env` for a Claude + Gemini setup:

```
DATABASE_URL=postgresql://agent:agentfactory@localhost:5432/agent_factory
DISCORD_BOT_TOKEN=your_token_here
DISCORD_TASK_CHANNEL_ID=your_channel_id
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
CLAUDE_BIN=/usr/local/bin/claude # find yours: which claude
GEMINI_BIN=/opt/homebrew/bin/gemini # find yours: which gemini
AGENT_ROUTER_ENABLED=1
```

6 Start & test

```
npm start
# You should see: [discord-bot] logged in as YourBotName#1234
```

In your Discord channel, type: `What is the capital of France?` — the bot reacts ⚡ and replies within ~30 seconds.

Then try a debate:

```
!debate: Should I use Postgres or SQLite for a small side project?
```

Watch the per-round embeds appear. After ~2 minutes you'll get an embed with  **Approve & Execute** /  **Reject** /  **Ask Claude** buttons.

7 Discord command reference

Command	What it does
<code>What is X?</code>	Auto-routed to best available agent (local model if possible, Claude as fallback)
<code>!claude: prompt</code>	Force Claude CLI
<code>!gemini: prompt</code>	Force Gemini CLI
<code>!local: prompt</code>	Force local Ollama model
<code>!codex: prompt</code>	Queue for next Codex session (async — no auto-reply)
<code>!debate: topic</code>	2-agent debate with default agents, one plays Red Team
<code>!debate claude vs gemini: topic</code>	Explicit agents — debate between Claude and Gemini
<code>!debate claude vs gemini 3: topic</code>	3 cycles each (more rounds, more thorough)
<code>!debate claude vs gemini --red gemini: topic</code>	Explicitly set Gemini as Red Team (adversarial role)

How debates work

Each agent takes turns responding. The **Red Team agent** is given adversarial instructions — its job is to attack the dominant view and find weaknesses, not to be balanced. The synthesizer (Claude) then weighs in, noting which Red Team objections were valid vs. weak. Consensus reached *under adversarial pressure* is treated as stronger signal.

Smart routing — saves money

Task type	Routes to
<code>summarize</code> , <code>commit_msg</code> , <code>boilerplate</code> , <code>explain_fn</code> , <code>seed_data</code>	Local model (free) → Claude as fallback
<code>long_doc</code> , <code>vision</code> , <code>pdf_analysis</code>	Gemini CLI
<code>chat</code> , everything else	Claude CLI

8 Troubleshooting

Symptom	Fix
Bot doesn't react to messages	Check Message Content Intent is enabled in Developer Portal (Bot tab → Privileged Gateway Intents). This is the #1 mistake.
<code>Used disallowed intents</code> error in logs	Same fix — enable Message Content Intent in portal, restart the service.
Bot joined server but channel ID not matching	Confirm Developer Mode is on (User Settings → Advanced → Developer Mode), then right-click channel again to copy ID.
Database connection refused	If using Docker: <code>docker compose ps</code> — ensure db shows healthy. If Supabase: check password in connection string.
Claude agent fails with auth error	Run <code>claude auth</code> in terminal to re-authenticate.
Gemini agent fails	Run <code>gemini auth</code> to re-authenticate with Google account.
Debate times out after 2 min	Local models (Ollama) are slow. Use <code>!debate claude vs gemini:</code> for fast debates.
Bot reacts ⚡ but no reply after 2 min	Check logs (<code>npm start</code> output). Debate results appear as webhook embeds, not bot replies.

Advanced: project context injection

Add a `CONTEXT_HINTS_JSON` env var so agents automatically receive context about your projects when a topic matches:

```
CONTEXT_HINTS_JSON=[
  {"pattern":"myapp","context":"MyApp is a SaaS tool for ..."},
  {"pattern":"backend","context":"Our backend is Node.js + Postgres on Railway ..."}
]
```

When you type `!debate: should myapp use Redis?`, the context is prepended automatically.

Auto-start on boot

macOS: Run `/setup-agent-factory` in Claude Code and ask it to set up the launchd service — it will write and load the plist automatically.

Linux: Ask Claude: "Set up a systemd service to run npm start in /path/to/agent-factory on boot."

Agent Factory · github.com/LNS-Research/ai-skills Run `/setup-agent-factory` in Claude Code for
· agent-factory/ interactive guided setup